SPHERE WEB AUTOMATION

Create Account Script Step-by-Step Walkthrough

A screen-by-screen, line-by-line map of how `tests/create-account.spec.ts` drives the Sphere signup journey — every navigation, selector, wait, assertion and fallback, with flow diagrams.

Script tests/create-account.spec.ts · 936 lines · 23 tests in 6 sections

Target https://sphere.aastrika.org (staging)

Run npm run run:create-account

Date 12 September 2026

Contents

1	What this script does	Purpose, scope, entry points
2	Configuration and inputs	Constants, env vars, generated identities
3	Master execution flow	Diagram — how the 6 sections chain
4	The UI journey	Diagram — every screen and branch
5	Section 1 — Select Language	7 steps
6	Section 2 — Form validation	5 steps
7	Section 3 — Full signup, OTP path	5 steps
8	The OTP subsystem	Diagram — 3-layer fallback + throttling
9	Section 4 — Phone + password path	1 step, 8 sub-steps
10	Section 5 — Sign up and accept T&C	2 steps — mints the reusable user
11	Section 6 — Responsive layout	3 steps
12	Artifacts the run writes	Files, sessions, hand-offs
13	Failure modes and fixes	Troubleshooting reference

How to read this document

Sections 5 through 11 walk the script in execution order. Each one opens with what the section proves, then a numbered step table giving the **action**, the **selector or API** it uses, the **wait** it applies, and the **assertion** that must hold. Selectors are quoted exactly as written in the spec, so you can search the file for any of them.

> **One tab, many tests**
> Sections 1–3 deliberately reuse a single browser tab (`sharedPage`) so the language screen is opened once and the form state carries forward. Sections 4–6 open their own tabs or contexts, because they need clean, unconfigured starting states.

1 What this script does

It is the end-to-end proof that a brand-new person can create a Sphere account unaided — and it is the factory that mints the fresh, logged-in users every other suite depends on.

The script exercises the complete public signup journey: **Select Language → Personal Details → OTP verification → Name confirmation → Terms & Conditions → logged-in Home**. It covers both login methods the portal offers (OTP and password), validates the form before submitting it, and checks the screens render on phone, tablet and desktop.

The OTP is not stubbed. The script registers a real `@yopmail.com` address, then reads the real verification mail out of that inbox, so the mail-delivery leg of the journey is covered too.

The three jobs it performs

Validates

- Language screen renders, is single-choice, gates Next
- Form gates Next until filled
- Field accepts email *and* phone
- Layout survives 390px → 1440px

Proves end-to-end

- Email + OTP signup reaches Home
- Phone + password signup completes
- Mail actually arrives and carries a usable OTP
- T&C accept unlocks the app

Produces

- `.last-registered-email.json`
- `.auth/fresh-user.json` — live session
- `.auth/fresh-user-email.json`
- Console trail of every email used

Where it runs, and why not localhost

The spec pins its own default target instead of inheriting the shared `BASE_URL`:

```
// tests/create-account.spec.ts:22
const BASE_URL = process.env.BASE_URL ?? 'https://sphere.aastrika.org';
```

Signup cannot run against the localhost prod-mirror — that mirror has no mail path and no user-provisioning backend, so an OTP would never arrive. This flow therefore targets staging by default. Passing `BASE_URL` still overrides it if you need to point somewhere else.

Entry points

Command	What it runs
`npm run run:create-account`	All 6 sections, headed, against staging. The full walkthrough this document describes.
`npm run mint:fresh-user`	Section 5 only — the fastest way to mint one new logged-in user and save its session for a course run.
`npm run report:auth`	Runs this spec alongside login, positive-flow and OTP specs headless, into a merged blob report.
`npm run compare:signup`	Replays the signup journey against two environments and diffs them.

> **Run it from the right folder**
> Every Playwright command must be issued from `SphereWebAutomation\`, not the `D:\Automation` root. Run from the root and every spec fails to resolve.

2 Configuration and inputs

2.1 Hard-coded constants

Constant	Value	Role in the flow
`CREATE_ACCOUNT_URL`	`{BASE_URL}/app/create-account`	The single entry URL every section navigates to.
`EMAIL_RECORD_FILE`	`.last-registered-email.json`	Where the most recent signup email is persisted for later specs.
`DEFAULT_PHONE`	`9999999999`	Seed for the phone path; decremented on collision.
`DEFAULT_PASSWORD`	`Sunbird@123`	Password used on the password path.
`FIRST_NAME / LAST_NAME`	`Test / Aastrika`	Identity typed into every signup form.
`DUMMY_PHONE`	`8971159809`	Documented for *manual* SMS testing only. Automation never uses it.

2.2 Environment variables

Variable	Default	Effect
`BASE_URL`	`sphere.aastrika.org`	Target environment.
`OTP_API_URL`	`(empty)`	Layer 3 OTP endpoint. Without it, Layer 3 is unavailable.
`OTP_EXTRACTION_KEY`	`(empty)`	Auth key for that endpoint.
`YOPMAIL_MIN_GAP_MS`	`6000`	Minimum gap between *any* two yopmail requests, run-wide.
`YOPMAIL_POLL_MS`	`7000`	Pause between inbox poll attempts.
`YOPMAIL_RETRIES`	`8`	Poll attempts per OTP fetch.
`FETCH_CYCLE_GAP_MS`	`12000`	Minimum gap between two whole OTP-fetch cycles.
`DUMMY_PHONE`	`8971159809`	Overrides the documented manual-test number.

2.3 How a fresh identity is generated

```
// :38 — a new inbox on every call, 90,000 possible addresses
function generateEmail() {
  const suffix = Math.floor(10000 + Math.random() * 90000);
  return `testautomation${suffix}@yopmail.com`;   // e.g. testautomation48213@yopmail.com
}

// :43 — phone path walks DOWN from 9999999999 on each collision
function generatePhone(seed = 0) {
  return String(parseInt(DEFAULT_PHONE) - seed);    // 9999999999, 9999999998, …
}
```

> **Why email is the automation identity, not phone**
> A phone number can register exactly one account, its SMS OTP cannot be read by automation, and a real-looking number risks texting a stranger. A yopmail address gives an unlimited supply of single-use identities whose OTP the script *can* read — which is precisely how the "a number can never be reused" limitation is overcome.

2.4 The seven shared helpers

Helper	Line	What it guarantees
`goToCreateAccount`	`:56`	Lands on the signup page — and if the app redirected to `/public/login`, clicks "Create an account" to get back on course.
`selectLanguageByIndex`	`:68`	Picks a radio by position through three escalating strategies, returning whether it actually got checked.
`advancePastLanguageScreen`	`:101`	Picks English *by label* (never by position) and clicks Next.
`formInputs`	`:120`	The three visible text inputs, excluding radio, hidden and password fields.
`clearFormInputs`	`:124`	Empties those three fields so a test starts from a known blank state.
`isOnTerms`	`:137`	Detects the T&C screen by route or by the Decline+Accept control pair — never by wording.
`isAlreadyExistsToast`	`:152`	Reads the snackbar and reports whether the identity was rejected as already registered.

3 Master execution flow

Six sections run in file order. Sections 1–3 share one tab; 4, 5 and 6 open their own.



Figure 1 — Execution order. Sections 1–3 operate on one tab; the boxed notes mark where a real account comes into existence and which files are written.

> **Serial by design**
> Sections 1–5 are declared `test.describe.serial` with `retries: 0`. Each step leaves the browser on the screen the next step expects, so parallelism or a retry would restart mid-journey on the wrong screen. A failure inside a serial block skips the rest of that block — which is correct: there is nothing to test on a screen you never reached. Section 6 is a plain `describe`, so its three viewport tests can run in parallel.

4 The UI journey

The screens the script actually walks through, and the two branches it takes.

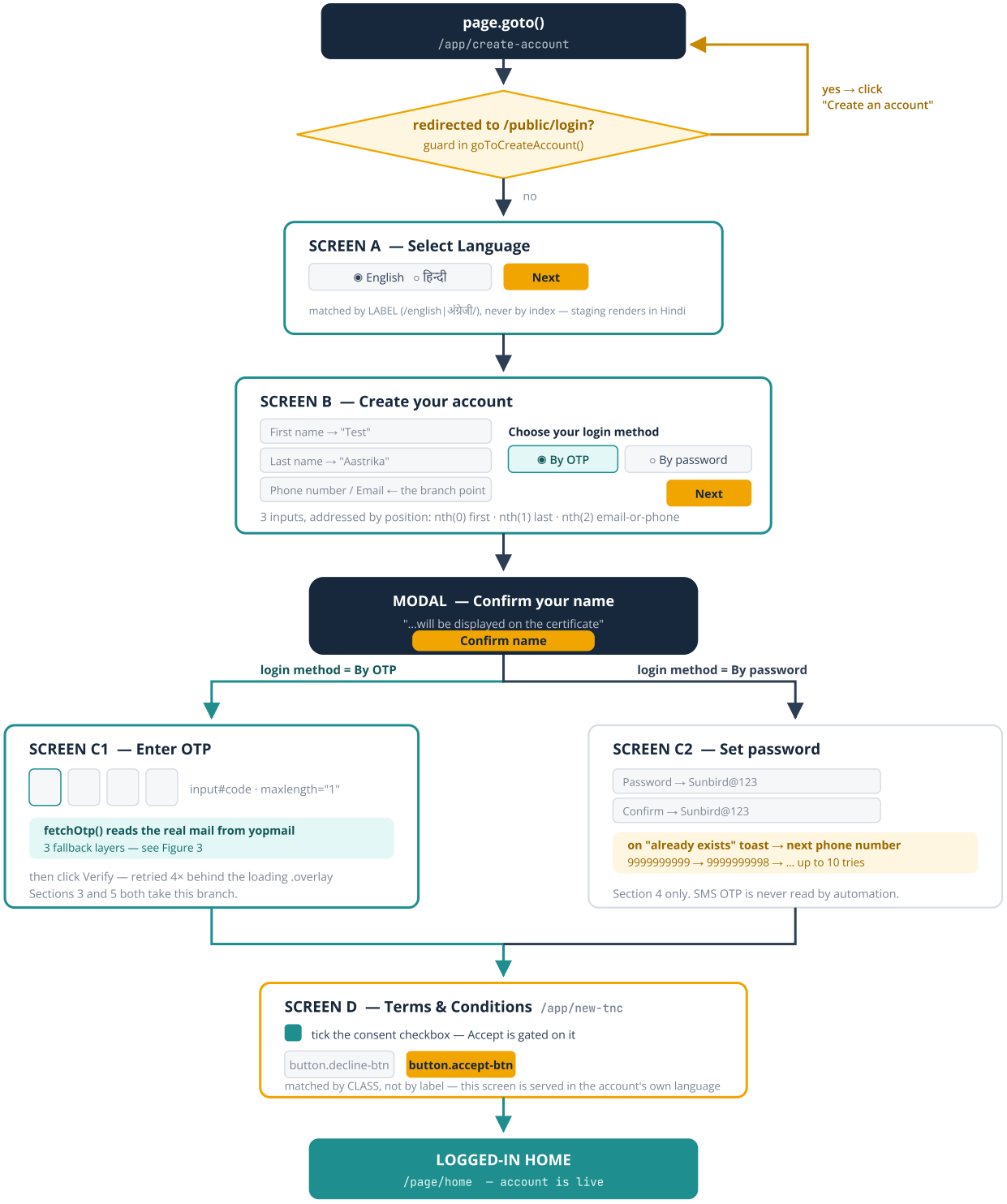


Figure 2 — The signup journey as the script walks it. The diamond is the only conditional navigation; the split after the confirm-name modal is decided by which login method was clicked on Screen B.

5 Section 1 — Select Language

`Step 1: Select Language` · 7 tests · line 428 · shared tab · serial

What it proves: the first screen a new user ever sees is complete and behaves like a proper single-choice control — it renders both languages, refuses to advance until one is chosen, and enables Next the moment one is.

Before the first test — the setup hook

```
// :416 — runs once, before every test in the file
test.beforeAll(async ({ browser }) ⇒ {
  sharedContext = await browser.newContext();   // clean cookie jar, no stored auth
  sharedPage    = await sharedContext.newPage(); // the one tab Sections 1-3 share
});
```

A fresh context matters: if the run inherited a logged-in session, `/app/create-account` would bounce straight to the home page and every step below would fail on a screen it never reached.

Step-by-step

#	Action	Selector / call	Wait & assertion
1	**Open Create Account page** Navigate to the signup entry URL, absorbing the login redirect if the app issues one.	`goToCreateAccount(sharedPage)` → `page.goto(CREATE_ACCOUNT_URL)` → `if /\/public\/login/ → click "create an account"`	waitForLoadState('networkidle') then 500 ms settle; 800 ms more after the redirect click. assert `mat-radio-button` count ≥ 2 — at least two languages offered.
2	**Heading + logo visible** Confirm the page is branded and titled, not a blank shell.	`getByText(/select language/i).first()` `img[alt*="logo" i], img[alt*="aastrika" i]`	Playwright auto-wait on `toBeVisible()`. assert both the heading and the logo image are visible.
3	**Next is disabled before any choice** The gate must hold on arrival.	`getByRole('button', { name: /^next$/i })` `.isDisabled()`	No extra wait — the page already settled in step 1. assert `isDisabled() === true`.
4	**English (index 0) is selectable** Click the first radio and verify it really took.	`selectLanguageByIndex(page, 0)` — see the three-strategy note below	600 ms after the click, then re-reads `input[type=radio]` checked state. assert the helper returns `true`.
5	**Hindi (index 1) is selectable** The second option must work the same way.	`selectLanguageByIndex(page, 1)`	Same 600 ms + verification cycle. assert returns `true`.

6	**Single-choice behaviour**
	Selecting English must clear Hindi — this is a radio group, not a checkbox list.
		`selectLanguageByIndex(page, 0)`, then read
		`input[type=radio].nth(0).isChecked()`
		`input[type=radio].nth(1).isChecked()`
			Reads both states immediately after selection.
			assert index 0 is `true` **and** index 1 is `false`.

7	**Next enables after a choice** The gate opens — English is left selected, which is the state Section 2 starts from.	`getByRole('button', { name: /^next$/i })`	`toBeEnabled({ timeout: 5000 })` — polls for up to 5 s while Angular updates the form state. assert the button is enabled.

Inside `selectLanguageByIndex` — three escalating strategies

Angular Material radios are not plain inputs; the real `<input>` sits under a styled wrapper that can swallow a click. The helper therefore escalates, verifying after each attempt and stopping as soon as the input reports checked:

1. **Click the wrapper.** `mat-radio-button` at the index, scrolled into view, with `{ force: true }`. Wait 600 ms, then read `input[type=radio].nth(idx).isChecked()`. If true, done.
2. **Click the inner ripple.** Falls through to `.mdc-radio` inside that wrapper — the element Material actually binds the handler to. Wait 500 ms, re-check.
3. **Drive the DOM directly.** A `page.evaluate` clicks the raw input, and if it still is not checked, sets `.checked = true` and dispatches `input` and `change` events so Angular's form control notices.

The helper returns the final checked state rather than throwing, which is why each test asserts on its return value instead of trusting the click.

> **Index is fine here, label is required later**
> These tests use index deliberately — they are testing radio *mechanics*, so position is the right handle. But `advancePastLanguageScreen` (:101), which picks the language the account is actually created in, matches on `/english|अंग्रेजी/i` instead. Staging renders this screen in Hindi, so a reordered list would silently sign the user up in Hindi and break every English selector downstream.

6 Section 2 — Form validation

`Step 2: Form Validation (shared tab)` · 5 tests · line 481 · shared tab serial

What it proves: the details form is complete, labelled, gated while empty, and flexible enough to accept either an email address or a phone number in its single identity field. Nothing here creates an account — Next is clicked exactly once, to move off the language screen.

Step-by-step

#	Action	Selector / call	Wait & assertion
1	**Advance to the form screen** Clicks the Next that Section 1 step 7 just enabled, with English still selected.	`getByRole('button', { name: /^next$/i }).click()`	waitForLoadState('networkidle') (errors swallowed) + 1200 ms for the screen transition. assert /create your account/i is visible within 8 s.
2	**All labels and options visible** Six checks in one test: the three field labels plus the login-method block.	`/first name/i` · `/last name/i` `/phone number.*email/i` `/choose your login method/i` `/by otp/i` · `/by password/i`	Auto-wait per `toBeVisible()`. assert all six are visible. The combined `/phone number.*email/i` label is what documents that one field takes both.
3	**Next disabled with empty fields** Blanks the form first so the state is unambiguous.	`clearFormInputs(page)` — `.fill('')` on the first 3 of `input:not([type=radio]):not([type=hidden]):not([type=password])`	100 ms between each clear, then 400 ms for validation to run. assert Next reports disabled.

4	**Form accepts text input**
	Types a full identity and reads each value back.
		`inputs.nth(0)` ← `"Test"`
		`inputs.nth(1)` ← `"Aastrika"`
		`inputs.nth(2)` ← `generateEmail()`
			No explicit wait — `fill()` is synchronous against the DOM.
			assert all three `inputValue()` round-trip exactly. This email is throwaway; it is never submitted.

5	**Same field accepts a phone number** Clears field 3 and types digits instead.	`inputs.nth(2).fill('')` then `.fill('9999999999')`	Immediate read-back. assert `inputValue() === '9999999999'` — no format mask rejects or rewrites it.

Why the inputs are addressed by position

```
// :120 — the three visible text fields, in DOM order
function formInputs(page) {
  return page.locator('input:not([type="radio"]):not([type="hidden"]):not([type="password"])');
}
//   nth(0) → First name
//   nth(1) → Last name
//   nth(2) → Phone number / Email
```

The three exclusions are what make positional addressing safe. Dropping `radio` keeps the language and login-method controls out of the list; dropping `hidden` excludes Angular's internal fields; dropping `password` means the same helper still returns exactly three fields on the password screen, where two more inputs appear. `clearFormInputs` caps its loop at `Math.min(count, 3)` for the same reason.

> **Carry-over into Section 3**
> Section 2 ends with the tab parked on a filled-in form. Section 3 does not assume that — it re-checks that `/create your account/i` is visible and re-navigates from scratch if it is not, so the two sections can also be run independently with `--grep`.

7 Section 3 — Full signup, OTP path

`Step 3: Full Signup — OTP Path` · 5 tests · line 534 · creates a real account · shared tab

What it proves: a brand-new person can register with an email address, receive a real verification mail, enter the code, and land inside the app. This is the first section that changes state on the server.

Step-by-step

#	Action	Selector / call	Wait & assertion
1	**Submit the form with By OTP + email** Self-heals its starting position, generates and records the identity, fills, picks OTP, submits. timeout 120 s	`if page.isClosed() → new page + goToCreateAccount + advancePastLanguageScreen else if !/create your account/i → same recovery clearFormInputs → nth(0,1,2) ← Test / Aastrika / signupEmail getByText(/by otp/i).first().click() Next`	400 ms after picking the login method; 2500 ms after Next for the modal to mount. effect `saveEmail()` writes `.last-registered-email.json` **before** submitting, so the address is recoverable even if the rest of the run dies.
2	**Name confirmation modal** Sphere prints this name on certificates, so it asks the user to confirm it.	`getByText(/confirm your name|will be displayed on the certificate|edit name|confirm name/i) then getByRole('button', {name:/confirm name/i}) .or(getByText(/confirm name/i))`	5 s visibility poll; 3500 ms after the click for the OTP request to fire and the next screen to render. assert the modal was shown.
3	**OTP screen is visible** Confirms the app moved to verification — tolerant of eight different wordings.	`Loops 8 regexes: /enter.*otp/i, /verification code/i, /verify/i, /one.time.*password/i, /enter.*code/i, /we (have|'ve) sent/i, /sent.*to.*your.*(email|inbox)/i, /check your email/i Fallback: count of input[maxlength="1"] ≥ 4`	2000 ms settle + scroll to top; 2 s per regex, first match wins. assert one marker matched. On failure it logs the URL and the first 200 chars of body text, so the report shows which screen it actually landed on.
4	**Retrieve and submit the OTP** The only step that leaves the application to read an external mailbox. timeout 180 s	`fetchOtp(signupEmail, { context: sharedContext }) enterOtp(page, otp) → input#code then re-fill across input[maxlength="1"] boxes getByRole('button', {name:/verify|submit|next/i})`	See Section 8 for the fetch timings. 150 ms per digit; 1000 ms before clicking Verify; 3000 ms after. assert an OTP came back. The verify click is retried up to 4× behind the loading overlay.
5	**Account created — lands on T&C or Home** Accepts either landing, because a first-time account is sent to the terms page while a returning one goes straight home.	`URL /terms/i or getByText(/terms of use|terms and conditions/i) or URL /\/(page|app)\/home/i then checkbox + getByRole('button', {name:/accept|agree|continue/i})`	2000 ms settle; 400 ms after ticking consent; 3000 ms after Accept. assert `onTerms || onHome || /\/page\/|\/app\//` — the account exists and the app let it in.

The overlay-safe click

Submitting an OTP puts a full-screen loading `.overlay` over the page. It intercepts pointer events, so a plain `click()` throws "element intercepts pointer events" and the test fails on a flow that actually worked. The

script handles it with a four-attempt loop:

```
for (let i = 0; i < 4 && !submitted; i++) {
  await page.locator('.overlay').first()
    .waitFor({ state: 'hidden', timeout: 5000 }).catch(() => {});  // let it clear
  try { await verifyBtn.click({ timeout: 6000 }); submitted = true; }
  catch {
    // enterOtp() pressed Enter — the form may ALREADY be submitted,
    // in which case the button is gone. That counts as success.
    if (!(await verifyBtn.isVisible().catch(() => false))) { submitted = true; break; }
    await page.waitForTimeout(1000);
  }
}
```

Why the OTP is typed twice

`enterOtp` (:393) targets `input#code`: if there are as many fields as digits it fills one digit each and presses Enter on the last, otherwise it fills a single field and presses Enter. Step 4 then re-reads the page for `input[maxlength="1"]` boxes and fills them again. The duplication is deliberate — some builds render the boxes without the `#code` id, and re-filling an already-correct field is harmless, while a half-filled OTP is not.

> **Every test guards on a closed page**
> Tests 2–5 each open with `if (sharedPage.isClosed()) { test.skip(...); return; }`. In a serial block a crashed tab would otherwise turn one real failure into four confusing ones; skipping keeps the report pointed at the step that actually broke.

8 The OTP subsystem

The most failure-prone leg of the journey, and the one with the most engineering behind it.

The script cannot ask the server for the code — that would skip the mail-delivery leg it is meant to cover. So it reads the real inbox. Three independent readers are tried in order, and each one's failure is remembered so the final error names all three.

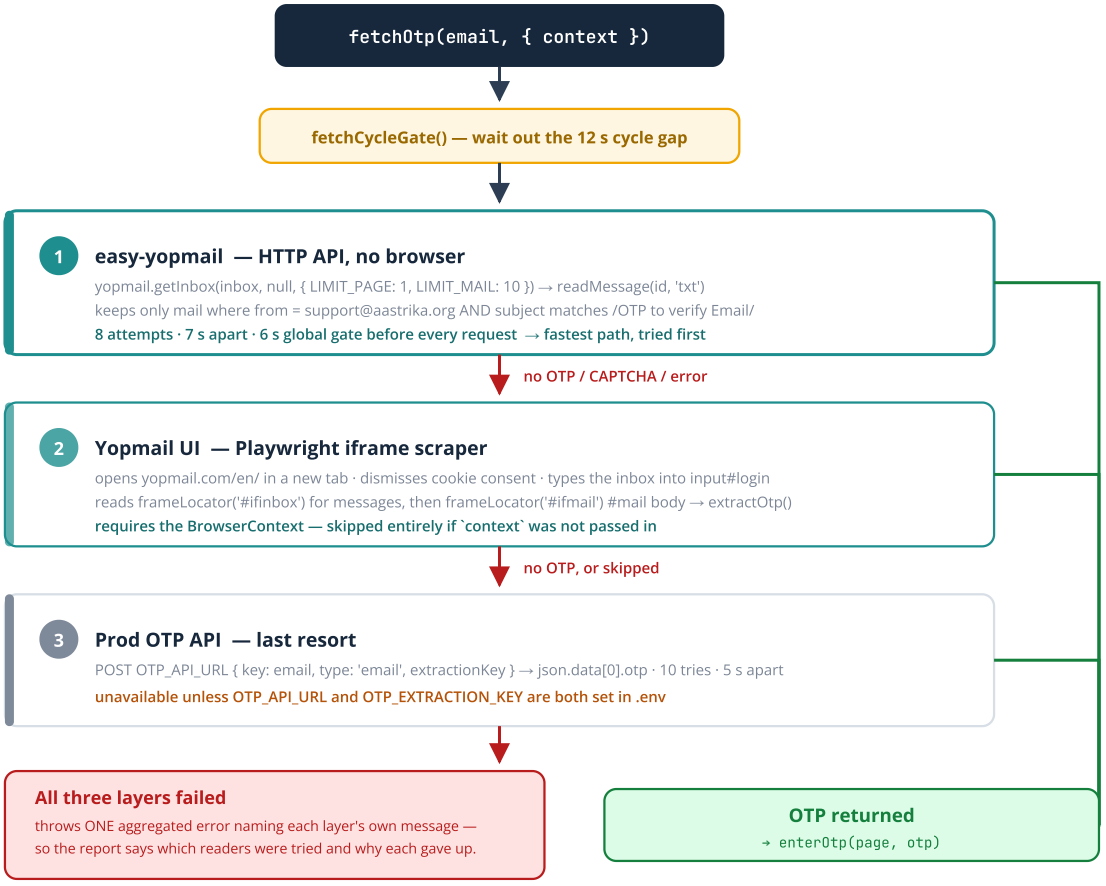


Figure 3 — Three readers, tried in order. Any layer returning a code short-circuits the rest; only an all-three failure raises, and it raises once with all three reasons.

8.1 How the code is recognised

Layer 1 filters on sender *and* subject before parsing, so a newsletter landing in the same inbox cannot be mistaken for a verification mail. The digits are then pulled with a two-stage regex that prefers the phrase form and only then falls back to any standalone 4–8 digit run:

```
const m = body.match(/is\s+(\d{4,8})/i) ?? body.match(/\b(\d{4,8})\b/);
```

Preferring `"…is 483921"` matters because mail bodies often carry other numbers — a year, a support phone, a ticket id — and a bare digit match would happily grab the wrong one.

8.2 Throttling — why the run deliberately goes slow

Yopmail does not rate-limit steady traffic; it flags **bursts** of automated inbox reads, and answers with a CAPTCHA that no layer can pass. Three independent gates keep the run under that threshold:

Gate	Default	What it spaces out
`YOPMAIL_MIN_GAP_MS`	`6 s`	**Every single yopmail request**, run-wide — each `getInbox`, each `readMessage`, each UI refresh click. Enforced by a module-level timestamp, so it holds across all signups in the run, not just within one.
`YOPMAIL_POLL_MS`	`7 s`	The pause between poll attempts while waiting for the mail to land.
`YOPMAIL_RETRIES`	`8`	How many attempts each layer makes before giving up — fewer attempts, spaced further apart.
`FETCH_CYCLE_GAP_MS`	`12 s`	The gap between two whole fetch cycles, which is the gap between two signups. Stops back-to-back registrations from bursting the inbox.

Both gates share one small primitive that simply sleeps for whatever remains of the minimum interval:

```
// :191
async function throttleGate(getLast, setLast, minGap) {
  const wait = minGap - (Date.now() - getLast());
  if (wait > 0) await new Promise((r) ⇒ setTimeout(r, wait));
  setLast(Date.now());
}
```

> **Budget the time**
> A full run signs up three times. With the defaults, expect roughly 30–90 s per OTP fetch depending on how fast the mail arrives — which is why the OTP steps carry 180 s timeouts. If you are debugging something unrelated, `--grep` to the section you care about rather than waiting out all three.

9 Section 4 — Phone + password path

`Step 4: Full Signup — Phone + Password Path` · 1 test · line 714 · creates a real account · timeout 180 s

What it proves: the second registration route works — identify by phone number, set a password instead of verifying a code. It runs in its own tab because it must start from the language screen with no carried-over form state.

Step-by-step within the single test

1. **Open a clean tab.** `sharedContext.newPage()`, then `goToCreateAccount(page)`. The tab shares the context's cookie jar but starts on a fresh navigation.
2. **Select English and advance.** `selectLanguageByIndex(page, 0)` then Next, `waitForLoadState('networkidle')` and 1500 ms.
3. **Enter the retry loop** over `seed = 0…9`. Each pass computes `generatePhone(seed)` — 9999999999, then 9999999998, and so on downward.
4. **Fill and submit.** nth(0) ← "Test", nth(1) ← "Aastrika", nth(2) ← the candidate phone. Click `/by password/i`, wait 400 ms, click Next, wait 2500 ms.
5. **Check the toast.** `isAlreadyExistsToast(page)` reads `mat-snack-bar-container, [role=alert], [class*=snackbar], [class*=toast]` and tests its text against `/already\s+(exists?|registered)|user.*exists?/i`. If it matches, `clearFormInputs` and continue with the next seed. Otherwise mark the phone accepted and break.
   assert `phoneAccepted === true` — all 10 candidates taken is a genuine failure worth reporting.
6. **Confirm the name.** Same modal as the OTP path, matched by `/confirm name/i` with a 5 s visibility poll. Optional here — it is clicked only if shown, then 2500 ms.
7. **Set the password.** Waits up to 10 s for `input[type="password"]` to become visible, fills it and its sibling `.nth(1)` with `Sunbird@123`, waits 800 ms.
8. **Submit and close.** Click Next, wait 4000 ms, log the final URL, then `page.close()` so the tab does not leak into Section 5.

> **Why this path is shallower than the OTP path**
>
> A phone number can register exactly one account — once 9999999999 is used, it is used forever. The script cannot read the SMS, and a real-looking number risks texting a stranger, so this section verifies that the password route *completes* and stops there. It does not assert a landing URL, only logs it. The deep, repeatable coverage lives on the email path, where identities are unlimited and the OTP is readable.

The `DUMMY_PHONE` constant at line 799 (`8971159809`) is a real dummy handset kept for *manual* phone testing, where a human reads the SMS. The script references it as `void DUMMY_PHONE` purely so the number stays documented in the file — automation never types it.

What a collision looks like in the console

```
⚠️  Phone 9999999999 already exists — trying next
⚠️  Phone 9999999998 already exists — trying next
✅ Phone 9999999997 accepted
✅ Confirmed name
✅ Password set
   Final URL after submit: https://sphere.aastrika.org/app/new-tnc
✅ Signup flow completed
```

10 Section 5 — Sign up and accept the T&C

`Step 5: New user signs up and ACCEPTS the T&C` · 2 tests · line 801 · creates a real account · writes the session hand-off

What it proves — and produces: this is the positive path end to end, and the section every other suite quietly depends on. It creates a brand-new user, accepts the terms, confirms the app let them in, and then **saves that live logged-in session to disk** so a course-completion run can pick up as this same fresh user.

> **Run it alone**
> `npm run mint:fresh-user` greps to `"Step 5"` — the fastest way to mint one logged-in user without paying for the other five sections.

Test 1 — Create a new user and reach the T&C page 180 s

#	Action	Selector / call	Wait & assertion

1	**New tab, new identity**	`sharedContext.newPage()`
		`tcEmail = generateEmail();`
		`saveEmail(tcEmail)`
			Email is recorded to disk before anything is submitted.

2	**Navigate and pick English by label**	`goToCreateAccount(tcPage)` `advancePastLanguageScreen(tcPage)`	Matches `/english|अंग्रेजी/i`, falls back to index 0 only if that is not visible. 1200 ms after Next.

3	**Fill the form, choose By OTP**	`clearFormInputs → nth(0,1,2)`
		`getByText(/by otp/i).first().click()`
		`Next`
			400 ms after the method click; 2500 ms after Next.

4	**Confirm the name**	`getByRole('button', {name:/confirm name/i}).or(getByText(...))`	6 s visibility poll, then 3000 ms. Clicked only if shown.

5	**Fetch, enter and submit the OTP**	`fetchOtp(tcEmail, { context: sharedContext })`
		`enterOtp(tcPage, otp)`
		`submitOtp(tcPage) — the local overlay-safe helper at :807`
			Up to 4 click attempts behind the overlay, 4000 ms after.
			assert an OTP was returned.

6	**Verify the T&C screen was reached**	`isOnTerms(tcPage)`	assert `onTerms === true` — a brand-new account must be shown the terms before anything else.

Test 2 — Accept the T&C and land on Home 150 s

#	Action	Selector / call	Wait & assertion
1	**Re-confirm position**	`isOnTerms(tcPage)`	assert still on the terms page before touching anything.
2	**Tick the consent checkbox**	`mat-checkbox, input[type=checkbox] — first, { force: true }`	400 ms. Accept is gated on this box, so it must be ticked first.
3	**Click Accept**	`button.accept-btn .or(getByRole('button', {name:/^accept$|^स्वीकार|^सहमत/i}))`	8 s visibility assertion, then click, then 4000 ms.
4	**Confirm the app let the user in**	`finalUrl + isOnTerms()`	assert `!stillOnTerms && /\/(page|app)\//i.test(finalUrl)` — off the terms page and inside the app.

#	Action	Selector / call	Wait & assertion
5	**Save the session hand-off**	`context.storageState({ path: '.auth/fresh-user.json' }) writeFileSync('.auth/fresh-user-email.json', { email, timestamp })`	Wrapped in try/catch — a write failure logs a warning but does not fail a signup that genuinely succeeded. Tab is then closed.

Two selector lessons, both learned the hard way

Match by class, not by wording

Portal language is a per-account setting, and staging serves this screen in Hindi, where Accept reads **स्वीकार करना**. An anchored `/^स्वीकार( करें)?$/` does not match that, so the button was never found and the terms were never accepted. `button.accept-btn` and `button.decline-btn` are identical in every language.

Detect the screen by route, not by text

Keying `isOnTerms` off the words "Terms of Use" produced false positives — that phrase also appears in the **home page footer**. The check now uses the route (`/new-tnc|\/tnc\b|terms-and-cond|consent/`) or the Decline + Accept control pair, which only ever coexist on the real terms screen.

Why the Decline detour was removed

An earlier version signed up, *declined* the terms, logged back in, and then accepted. It was dropped: it doubled the OTP fetches and therefore the runtime, and it made minting a fresh user fail whenever the public login page changed — without adding any coverage the positive path needs. Decline is now tested where it belongs, not on the critical path that every other suite depends on.

The hand-off, and who consumes it

```
// .auth/fresh-user.json       → full Playwright storageState (cookies + origins)
// .auth/fresh-user-email.json → { "email": "testautomation48213@yopmail.com", "timestamp": … }

// Consumed by tests/helpers/authedTest.ts when USE_FRESH_USER=1
```

The session is saved rather than the credentials, because a yopmail-email account is OTP-only — there is no password to log back in with. Capturing the live session is the only way a later suite can continue as this user.

11 Section 6 — Responsive layout

`Responsive Layout` · 3 tests · line 916 · parallel-safe · creates no account

What it proves: the two screens a user must get through before any account exists render correctly from a small phone up to a desktop. This is the only non-serial block in the file — each test builds its own context at its own viewport, so nothing is shared and the three can run together.

Viewport	Size	Represents
`Mobile`	`390 × 844`	The dominant real-world case — a modern phone. The narrowest layout the signup form must survive.
`Tablet`	`768 × 1024`	The breakpoint where most responsive grids switch between stacked and side-by-side.
`Desktop`	`1440 × 900`	The design reference width.

What each of the three tests does

1. **Build an isolated context at that viewport.** `browser.newContext({ viewport: { width, height } })` then `newPage()`. A fresh context per viewport is what makes the block parallel-safe.
2. **Navigate.** `goToCreateAccount(page)`, including the login-redirect guard.
3. **Clear the language screen.** `selectLanguageByIndex(page, 0)` then Next, `waitForLoadState('networkidle')` and 1200 ms.
4. **Assert the form rendered.** `expect(page.getByText(/create your account/i).first()).toBeVisible()` — proof that both screens survived the narrower viewport and the form is reachable, not clipped or collapsed.
5. **Tear down.** `ctx.close()` disposes the context and its page together.

> **Scope, honestly stated**
> This is a smoke check for reachability, not a visual-regression test. It confirms the layout does not break badly enough to hide the form; it will not catch a mildly misaligned label or an overlapping icon. Pixel-level checks belong in the brand and Figma comparison suites.

12 Artifacts the run writes

File	Written by	Contents and purpose
`.last-registered-email.json`	Sections 3 and 5, via `saveEmail()`	`{ "email": "…", "timestamp": … }`. Overwritten on every signup, so it always holds the most recent address. Written **before** the form is submitted, so the identity survives a crash mid-signup.
`.auth/fresh-user.json`	Section 5, test 2	A full Playwright `storageState` — cookies plus origin storage for the brand-new, terms-accepted user. The hand-off that lets a course suite continue as this user.
`.auth/fresh-user-email.json`	Section 5, test 2	Which email that session belongs to, plus when it was minted — so a stale session can be spotted rather than silently reused.
`generated-emails.txt` `registered-accounts.xlsx`	Reporters / helper scripts	The running ledger of identities created across runs. Useful when you need to find an account you made last week.
`playwright-report/` · `test-results/`	Playwright	HTML report, plus traces and screenshots for any failing test.

The console trail

The script logs a checkmarked line at every meaningful step, which makes a headless run readable without opening the report:

```
✅ Found 2 language options
✅ Heading and logo visible
✅ Next disabled before selection
✅ Single-choice behavior confirmed
✅ On Create Account form
💾 Saved email to .last-registered-email.json: testautomation48213@yopmail.com
✅ Form submitted with email: testautomation48213@yopmail.com
✅ Name confirmation modal shown
✅ OTP screen visible
==========================================
Starting OTP Fetch Process | Email: … | Use Yopmail: true
   [yopmail 1/8] inbox has 0 message(s)
   [yopmail 2/8] inbox has 1 message(s)
      from="support@aastrika.org" subject="OTP to verify Email"
✅ OTP retrieved: 483921
✅ Accepted T&C — logged in successfully
💾 Saved fresh-user session for course completion: testautomation48213@yopmail.com
```

Accounts created per full run

Section	Identity	Verification	Ends up
Section 3	`testautomationNNNNN@yopmail.com`	Email OTP	Terms accepted if the screen appeared; session not saved.
Section 4	`9999999999 (descending)`	Password	Submitted; landing URL logged only. Burns one phone number permanently.
Section 5	`testautomationNNNNN@yopmail.com`	Email OTP	Terms accepted, on Home, **session saved to disk**.

13 Failure modes and fixes

Symptom	Most likely cause	What to do
Every spec fails to resolve	Playwright was invoked from `D:\Automation` instead of `SphereWebAutomation\`.	Change into `SphereWebAutomation\` and re-run. This is the single most common cause of a whole-suite failure.
"OTP fetch failed across all layers"	Yopmail served a CAPTCHA after a burst, or the mail never arrived.	Read the aggregated error — it names all three layers' reasons. If Layer 1 mentions CAPTCHA, raise `YOPMAIL_MIN_GAP_MS` and `FETCH_CYCLE_GAP_MS` and re-run. Configure `OTP_API_URL` + `OTP_EXTRACTION_KEY` to make Layer 3 available as a real safety net.
Layer 2 logged as "skipped (no context)"	`fetchOtp` was called without `{ context }`.	Pass the BrowserContext. Both in-file call sites already do; a new call site must too, or you silently run on two layers instead of three.
"OTP screen not detected" with a URL and body preview logged	The app went somewhere else after Confirm name — often "already registered", or a server error on the signup POST.	Read the logged body preview; it usually names the real problem. If the wording simply changed, add the new phrase to the 8-marker list at line 594.
Verify click throws "intercepts pointer events"	The loading `.overlay` stayed up longer than the four retry attempts allowed.	The backend is slow. Confirm staging is healthy; the retry loop is already the correct pattern, so widen its timeout rather than removing the guard.
Section 5 fails with `onTerms === false`	Either the account did not actually get created, or the terms screen rendered in a language the text matcher missed.	Check the logged URL first. If it *is* `/app/new-tnc`, the class-based selectors are the fix — match `button.accept-btn`, never the label.
Section 4 fails with `phoneAccepted === false`	All 10 candidates from 9999999999 downward are already registered.	Expected eventually — the pool is consumed one number per run. Widen the loop or change the `DEFAULT_PHONE` seed to an unused block.
Language screen never appears; page lands on Home	The context carried a logged-in session, so the app redirected away from signup.	Confirm the run is not reusing a `storageState`. `beforeAll` creates a clean context precisely to prevent this.
The whole account got created in Hindi	Something selected index 0 on a screen where Hindi was listed first.	Use `advancePastLanguageScreen`, which matches `/english|अंग्रेजी/i` by label. Reserve `selectLanguageByIndex` for the radio-mechanics tests in Section 1.
Four tests fail where one broke	Normal serial behaviour — but check whether the tab crashed.	The `isClosed()` guards convert the downstream tests to skips. Fix the first failure; the rest usually follow.

Useful run variants

```
# one section only
npx playwright test tests/create-account.spec.ts --grep "Step 5" --headed

# slow yopmail right down when it starts rate-limiting
cross-env YOPMAIL_MIN_GAP_MS=12000 FETCH_CYCLE_GAP_MS=25000 \
  npx playwright test tests/create-account.spec.ts

# point at another environment
cross-env BASE_URL=https://your-env.example.org \
  npx playwright test tests/create-account.spec.ts --headed

# trace a failure end to end
```

```
npx playwright test tests/create-account.spec.ts --trace on
npx playwright show-trace test-results/<run>/trace.zip
```

The one rule to keep

Every selector in this script is chosen to survive a language change — matched by route, by class, or by a regex that covers both English and Hindi. When you add a selector, match the *structure*, not the words. The portal's language is a per-account setting, and the account this script creates may not be in yours.